IOT CONTROL

나 재 우 • 변 한 빛

Index

01 프로젝트 소개

02 설계

03 핵심 기능

04 트러블 슈팅

05 QA

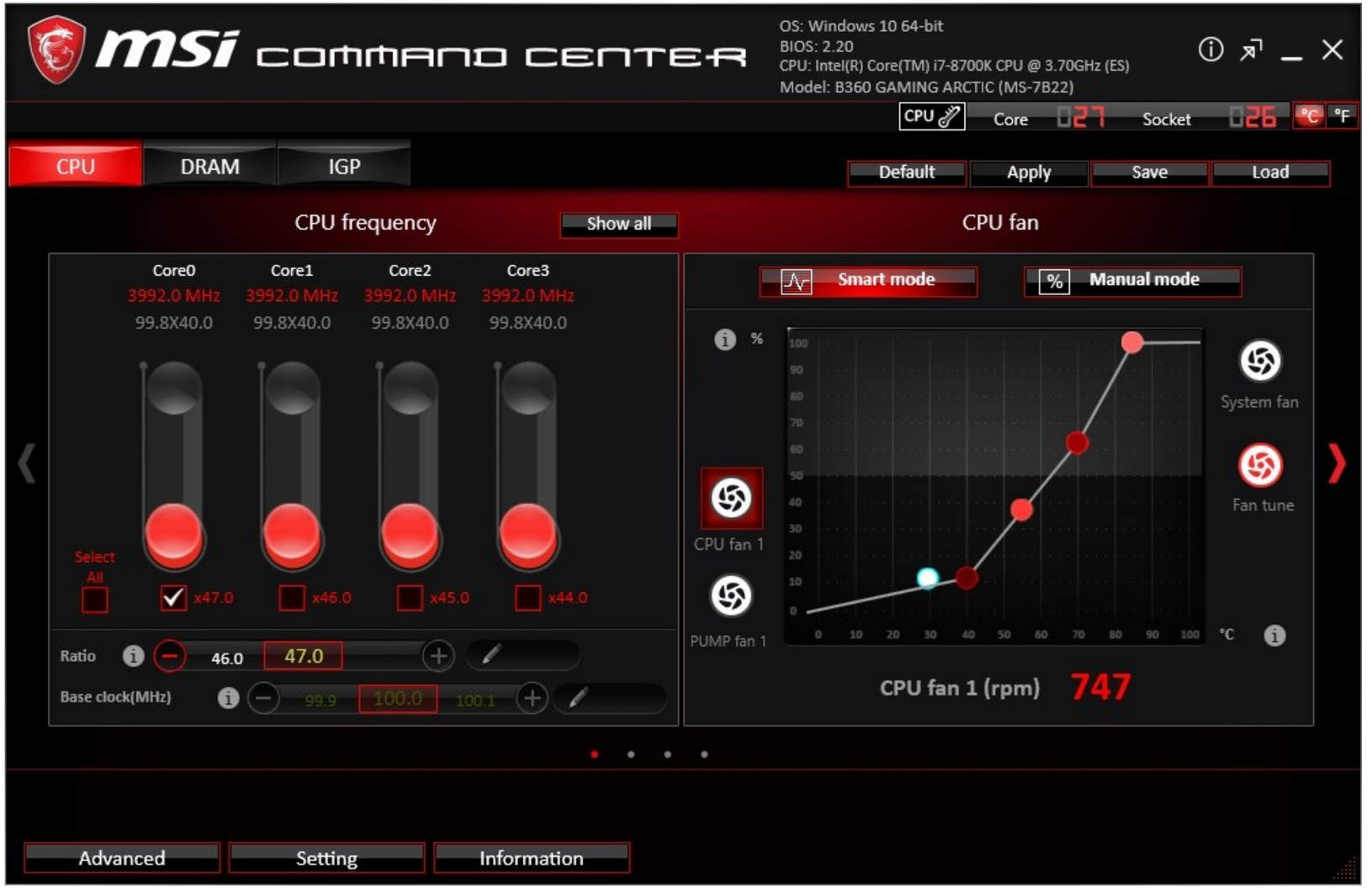
프로젝트 소개

누구든지 IoT를 쉽게 제작할 수 있도록

IoT Control은 복잡한 IoT 개발 과정의 진입장벽을 낮추기 위해
Raspberry Pi, STM32, Arduino를 통합한 **IoT Maker 플랫폼**입니다.

사용자는 다양한 IoT 장치를 손쉽게 연결하고 제어하여
자신만의 IoT 환경을 구축할 수 있습니다.

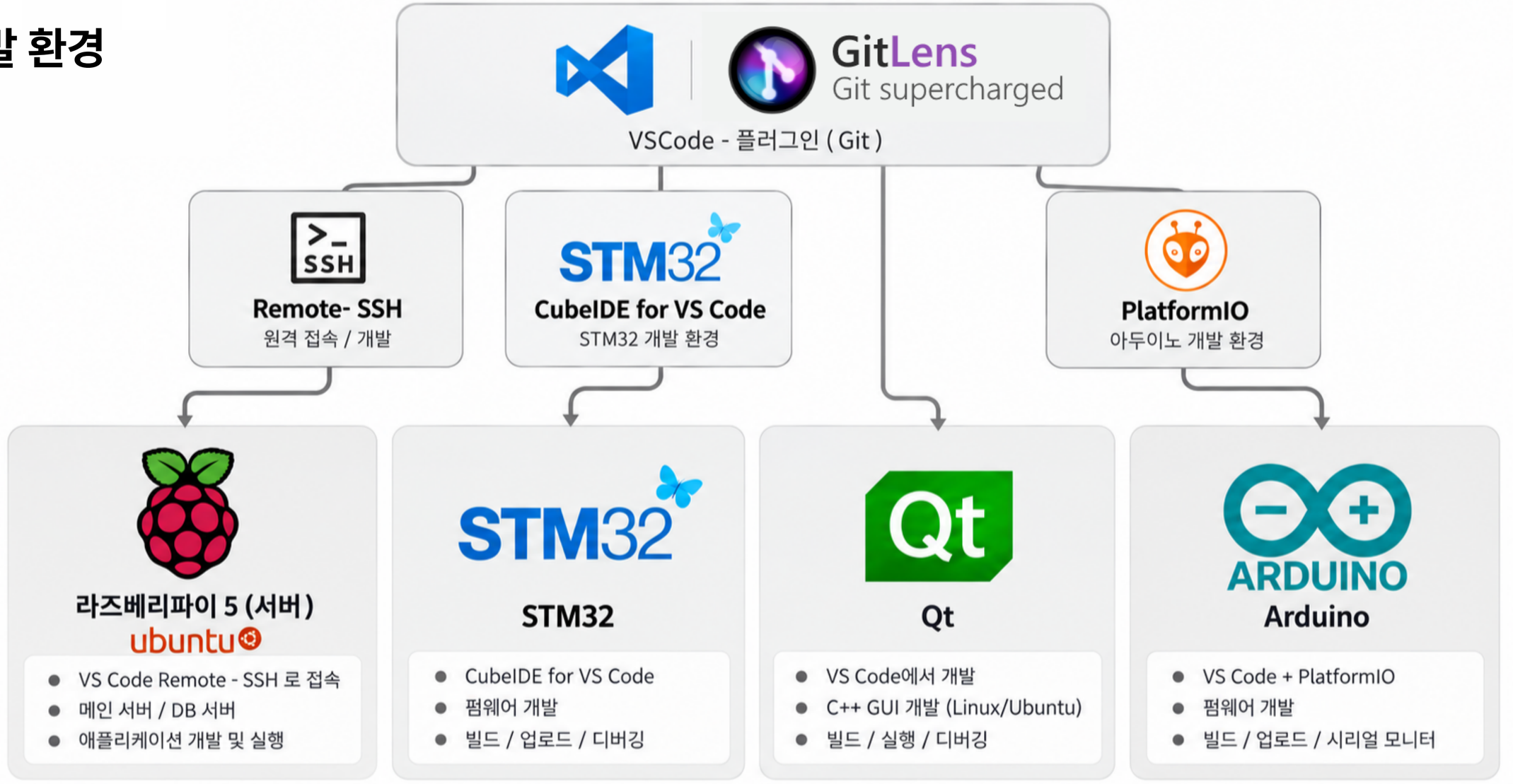



msi COMMAND CENTER
OS: Windows 10 64-bit
BIOS: 2.20
CPU: Intel(R) Core(TM) i7-8700K CPU @ 3.70GHz (ES)
Model: B360 GAMING ARCTIC (MS-7B22)
CPU
Core
Socket
°C
°F
CPU
DRAM
IGP
Default
Apply
Save
Load
CPU frequency
Show all
Core0
Core1
Core2
Core3
3992.0 MHz
99.8X40.0
Select All
x47.0
x46.0
x45.0
x44.0
Ratio
46.0
47.0
Base clock(MHz)
99.9
100.0
100.1
CPU fan
Smart mode
Manual mode
%
System fan
Fan tune
CPU fan 1
PUMP fan 1
°C
CPU fan 1 (rpm)
747
Advanced
Setting
Information

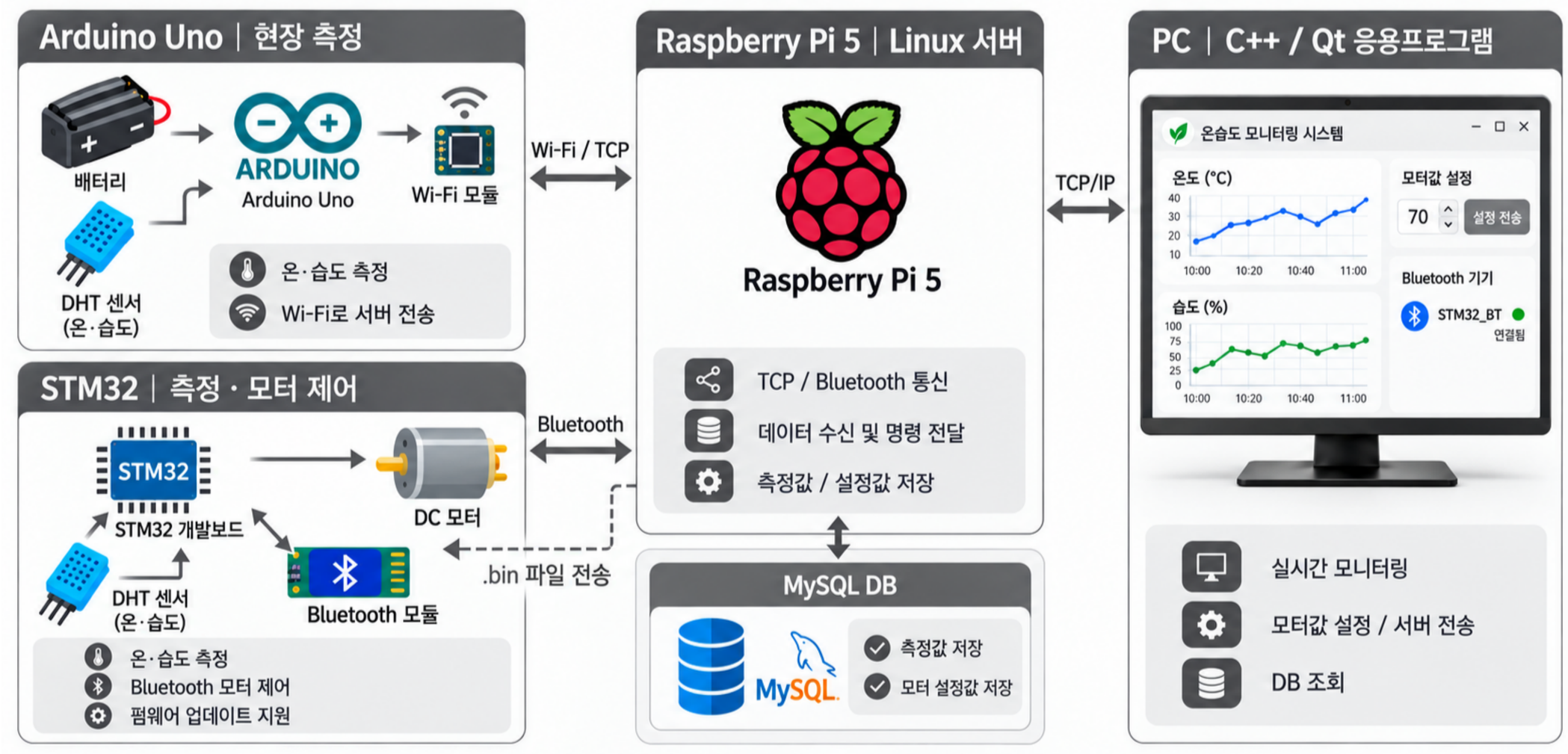
시 연

설 계

개발 환경


GitLens
Git supercharged
VSCode - 플러그인 (Git)
Remote- SSH
원격 접속 / 개발
CubeIDE for VS Code
STM32 개발 환경
PlatformIO
아두이노 개발 환경
라즈베리파이 5 (서버)
ubuntu
VS Code Remote - SSH 로 접속
메인 서버 / DB 서버
애플리케이션 개발 및 실행
STM32
CubeIDE for VS Code
펌웨어 개발
빌드 / 업로드 / 디버깅
Qt
VS Code에서 개발
C++ GUI 개발 (Linux/Ubuntu)
빌드 / 실행 / 디버깅
Arduino
VS Code + PlatformIO
펌웨어 개발
빌드 / 업로드 / 시리얼 모니터

시스템 아키텍처


Arduino Uno | 현장 측정
배터리
ARDUINO
Arduino Uno
Wi-Fi 모듈
DHT 센서
(온·습도)
온·습도 측정
Wi-Fi로 서버 전송
Wi-Fi / TCP
Raspberry Pi 5 | Linux 서버
Raspberry Pi 5
TCP / Bluetooth 통신
데이터 수신 및 명령 전달
측정값 / 설정값 저장
TCP/IP
PC | C++ / Qt 응용프로그램
온습도 모니터링 시스템
온도 (°C)
40
30
20
10
10:00
10:20
10:40
11:00
모터값 설정
70
설정 전송
Bluetooth 기기
STM32_BT
연결됨
습도 (%)
100
75
50
25
0
10:00
10:20
10:40
11:00
실시간 모니터링
모터값 설정 / 서버 전송
DB 조회
STM32 | 측정 · 모터 제어
STM32
STM32 개발보드
DC 모터
DHT 센서
(온·습도)
Bluetooth 모듈
Bluetooth
.bin 파일 전송
온·습도 측정
Bluetooth 모터 제어
펌웨어 업데이트 지원
MySQL DB
MySQL
측정값 저장
모터 설정값 저장

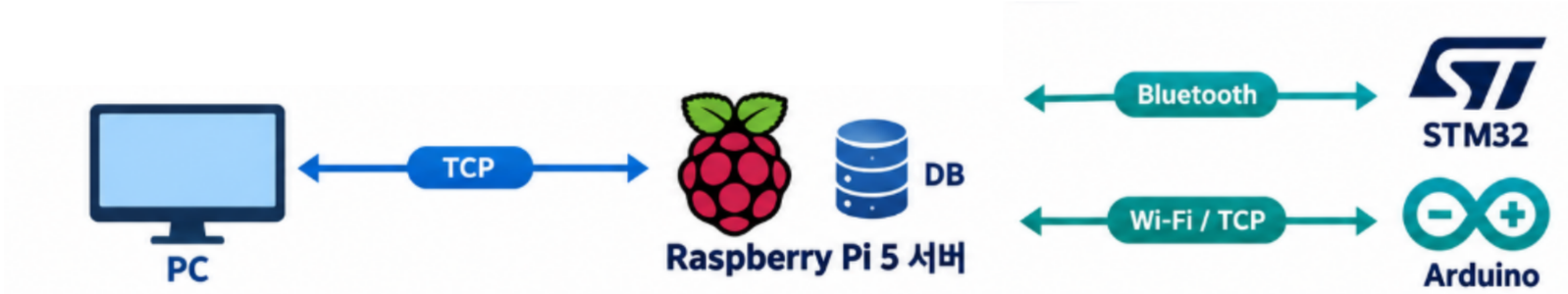
프로젝트 폴더 구조

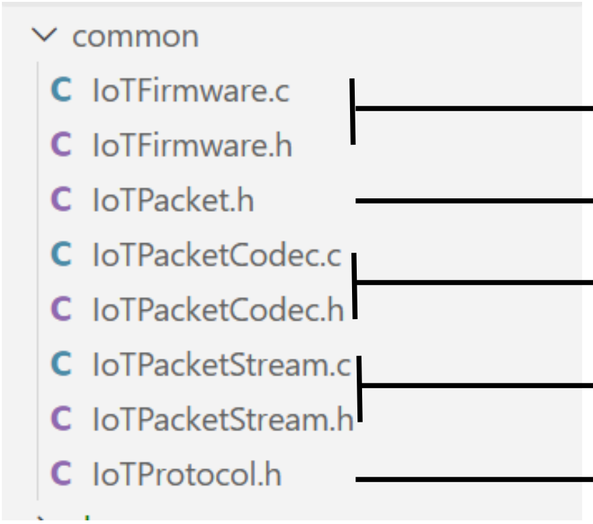
플랫폼별 코드와 공용 기능 분리



common 폴더

통합 통신 규약 및 바이너리 처리





Firmware 업데이트 관련 / STM32 플래시 메모리 / 파일 이미지 관리

페이로드 구조체

명령어 별 패킷 작업

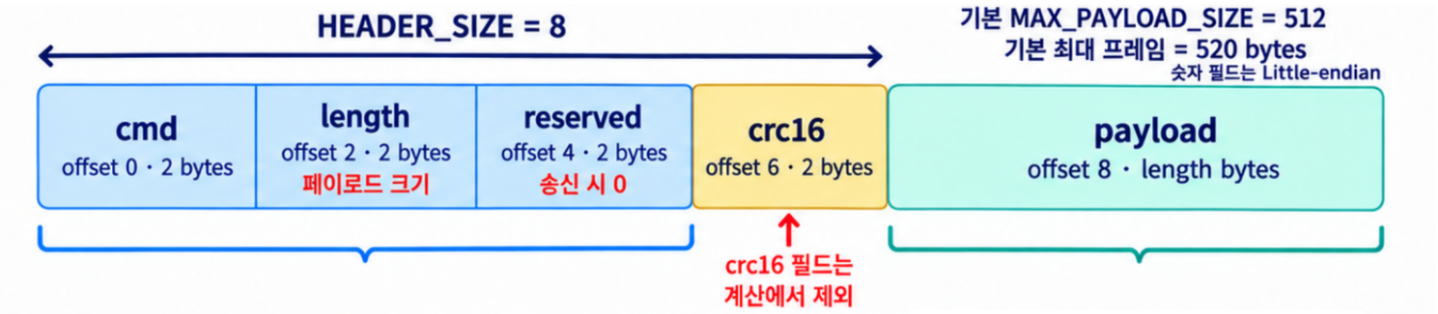
패킷 직렬화 / 역 직렬화 코드

헤더 정의 및 명령어 선언

패킷 구조 및 명령 비트

Little-endian 정렬

헤더 사이즈는 8바이트 고정




```c
typedef struct _HeaderData
{
    uint16_t cmd;
    uint16_t length;
    uint16_t reserved;
    uint16_t crc;
} HeaderData;

#define CMD_ACK_BIT 0x8000
#define CMD_NOTIFY_BIT 0x4000
#define CMD_ID_MASK 0x3FFF
```

```c
typedef struct _DhtData
{
    uint16_t temp;
    uint16_t humi;
} DhtData;

typedef struct _MemData
{
    /* Fixed-width fields;
    char id[MEM_ID_SIZE];
    char pw[MEM_PW_SIZE];
} MemData;

typedef struct _FanData
{
    uint16_t fanSpeed;
} FanData;
```

Header cmd 종류

메세지에 cmd를 선택하여 사용하여
통신 목적을 수신측에게 제공

```
/* Message IDs */
#define MSG_LOGIN 0x0001
#define MSG_CHAT 0x0002
#define MSG_DHT 0x0003
#define MSG_FAN 0x0004
#define MSG_CON 0x0005
#define MSG_BT_REGISTER 0x0006
#define MSG_BT_CONNECT 0x0007
#define MSG_DHT_ALL 0x0008
#define MSG_DHT_COLLECT 0x0009
#define MSG_FAN_QUERY 0x000A
#define MSG_FW_BEGIN 0x000B
#define MSG_FW_CHUNK 0x000C
#define MSG_FW_END 0x000D
#define MSG_FAN_UPDATE 0x000E
#define MSG_SESSION_ALL 0x000F
#define MSG_FAN_APPLY 0x0010
#define MSG_DHT_REFRESH 0x0011
#define MSG_BT_CONNECT_ALL 0x0012
#define MSG_BT_SCAN 0x0013
```

구조체 - > 바이트 변환 (직렬화)

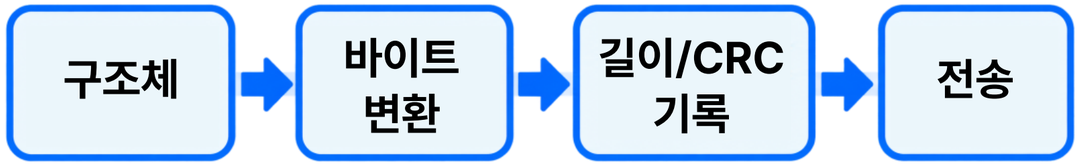
팬 속도 전달 예시

팬속도 60% 송신시

headerData 8 byte + fanData 2byte (Little-endian) 전송

{ headerData }{ (payload) 3C 00 }

송신




```
size_t MakeFanControlPacket(uint8_t *buffer, size_t size, const FanData *data)
{
    IoTPacketWriter writer;

    IoTPacketBegin(&writer, buffer, size, REQ_FAN);
    IoTPacketPushUint16(&writer, data->fanSpeed);
    return IoTPacketEnd(&writer);
}
```

```
void IoTPacketBegin(IoTPacketWriter *writer, uint8_t *buffer, size_t capacity, uint16_t cmd)
{
    writer->buffer = buffer;
    writer->capacity = capacity;
    writer->length = 0;
    writer->overflow = 0;
    IoTPacketPushUint16(writer, cmd);
    IoTPacketPushUint16(writer, 0); /* length, filled by IoTPacketEnd() */
    IoTPacketPushUint16(writer, 0); /* reserved */
    IoTPacketPushUint16(writer, 0); /* crc16, filled by IoTPacketEnd() */
}
```

```
void IoTPacketPushUint16(IoTPacketWriter *writer, uint16_t value)
{
    uint8_t bytes[2];

    bytes[0] = (uint8_t)(value & 0xFF);
    bytes[1] = (uint8_t)(value >> 8);
    IoTPacketPushBytes(writer, bytes, sizeof(bytes));
}
```

구조체 < - 바이트 변환 (역직렬화)

팬 속도 전달 예시

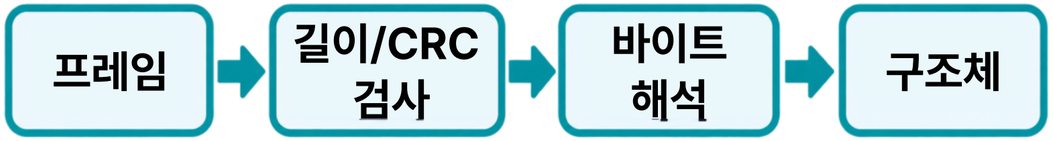
팬 속도 60% 수신시

headerData 8 byte + fanData 2byte (Little-endian) 수신

{ headerData }{ (payload) 3C 00 }

0×3C | (0×00 << 8) = 0×003C = 60

수신




```
void DecodePacketHeader(const uint8_t *buffer, HeaderData *header)
{
    header->cmd = GetUint16(buffer);
    header->length = GetUint16(buffer + 2);
    header->reserved = GetUint16(buffer + 4);
    header->crc = GetUint16(buffer + 6);
}
int CheckPacketLength(uint16_t cmd, size_t length)      Samana0104, 4일 전 • [fea
{
    switch(cmd)
    {
        case REQ_BT_CONNECT_ALL:
        case REQ_BT_SCAN: return length == 0 ? 0 : -1;
        case NFY_BT_CONNECT_ROW: return length == BT_CONNECT_ROW_SIZE ? 0 : -1;
        case NFY_BT_SCAN_ROW: return length == BT_SCAN_ROW_SIZE ? 0 : -1;

int CheckPacketCrc(const uint8_t *headerData, const HeaderData *header, const uint8_t *payload)
{
    uint16_t crc = UpdatePacketCrc(0xFFFF, headerData, CRC_COVERED_HEADER_SIZE);

    crc = UpdatePacketCrc(crc, payload, header->length);
    return crc == header->crc ? 0 : -1;
}

int ReadFanData(const uint8_t *payload, size_t length, FanData *data)
{
    IoTPacketReader reader;

    IoTPacketOpen(&reader, payload, length);
    data->fanSpeed = IoTPacketPopUint16(&reader);
    return IoTPacketCheckRead(&reader);
}

 int IoTPacketCheckRead(const IoTPacketReader *reader)
 {
     return !reader->underflow && reader->offset == reader->length ? 0 : -1;
 }
```

REQ / ACK / NFY 통신 흐름

REQ : 작업요청
ACK : 요청에 대한 처리 결과 응답
NFY : 센서 값 또는 조회 결과 알림

ACK 시 0×8000을 | 하여 제공

NFY 시 0×4000을 | 하여 사용

팬 속도 처리

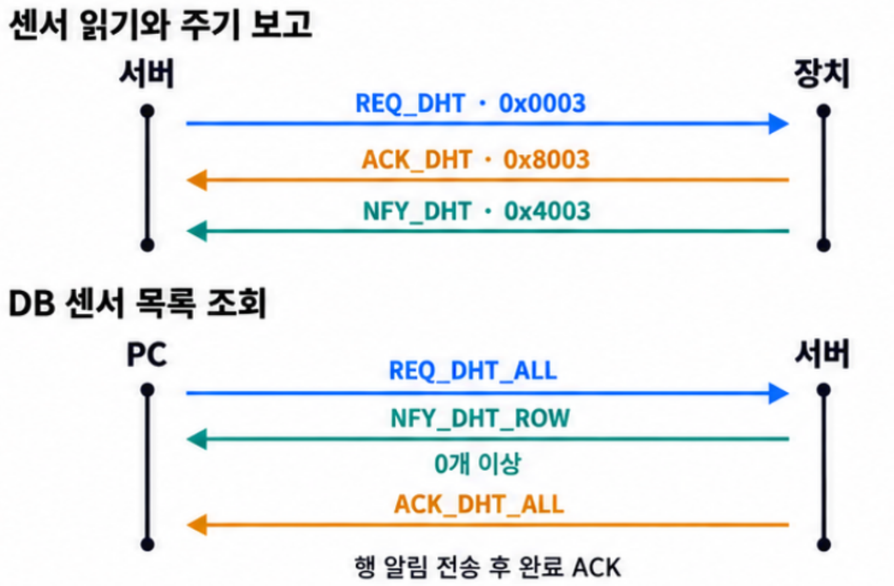
QUERY : DB 조회 / UPDATE : DB 저장
APPLY : 저장된 속도를 장치에 적용
장치 ACK 수신 후 PC에 적용 결과 응답

```
#define CMD_ACK_BIT 0x8000
#define CMD_NOTIFY_BIT 0x4000
#define CMD_ID_MASK 0x3FFF

#define REQ_TO_ACK(id) ((uint16_t)((id) | CMD_ACK_BIT))
#define ACK_TO_REQ(cmd) ((uint16_t)((cmd) & CMD_ID_MASK))
#define MAKE_NOTIFY(id) ((uint16_t)((id) | CMD_NOTIFY_BIT))
#define IS_ACK(cmd) (((cmd) & CMD_ACK_BIT) != 0)
#define IS_NOTIFY(cmd) (((cmd) & CMD_NOTIFY_BIT) != 0)

#define REQ_DHT_ALL MSG_DHT_ALL             /* no payload */
#define NFY_DHT_ROW MAKE_NOTIFY(MSG_DHT_ALL) /* DhtRowData */
#define ACK_DHT_ALL REQ_TO_ACK(REQ_DHT_ALL) /* ResultData */

#define RESULT_SUCCESS 0x00
#define RESULT_FAIL 0x01
```



```
        * Execute \help or \? for help;
sql> • SELECT * FROM `iot`.`fan`
```

singleton_id	speed
1	0

OK, 1 record retrieved in 5.541ms

STM32 펌웨어 업데이트

플레시 메모리 관리 및 이미지 관리

BEGIN : 크기 / CRC32 / 버전 전달, Staging 초기화
CHUNK : offset 순서대로 1 ~ 256 byte 전송
END : 검증 후 준비 표시, ACK 후 재부팅

Bootloader가 Staging → Application 복사 / 검증
복사 중 전원 차단 시 다음 부팅에서 재시도

```c
typedef struct _FirmwareBeginData
{
    uint32_t size;    /* image bytes, 1..FIRMWARE_APP_MAX_SIZE */
    uint32_t crc32;   /* ComputeFirmwareCrc32 of the whole image */
    uint32_t version; /* FirmwareInfo.version inside the image */
} FirmwareBeginData;

/* REQ_FW_CHUNK payload: offset, then length bytes (length comes from the frame length). */
Samana0104, 3일 전 | author (Samana0104)
typedef struct _FirmwareChunkData
{
    uint32_t offset;
    uint16_t length;
    uint8_t data[FIRMWARE_CHUNK_SIZE];
} FirmwareChunkData;

/* ACK_FW_CHUNK payload: offset echoes the acknowledged chunk. */
Samana0104, 3일 전 | author (Samana0104)
typedef struct _FirmwareChunkAckData
{
    uint8_t result;
    uint32_t offset;
} FirmwareChunkAckData;
```

업데이트



STM32F411 Flash · 512 KB

Sector 0 · 16 KB	0x08000000 · Bootloader
Sector 1–4 · 112 KB	0x08004000 · Application
Sector 5 · 128 KB	0x08020000 · Staging
Sector 6 · 128 KB	0x08040000 · 미사용
Sector 7 · 128 KB	0x08060000 · SData

Staging 이미지 +0x100 · SData 설정 보존

epoll 기반 서버 구축

Raspberry5 / Core / RNetwork.c
TCP / Bluetooth 연결을 공유 epoll에 등록

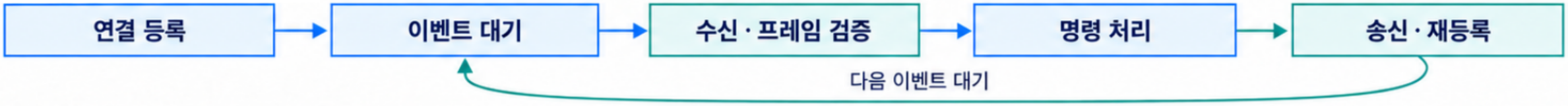
준비된 연결의 이벤트를 여러 워커가 처리

O-NONBLOCK .• 소켓 I/O 대기 방지
EPOLLONESHOT : 같은 연결의 중복 처리 방지
처리 후 EPOLL-CTL-MOD로 이벤트 재등록

수신 버퍼에서 프레임 조립 및 CRC 검사
명령 처리 후 응답 송신, 남은 데이터는 이어서 전송

처리 예시
PC의 조회 요청과 장치 센서 보고를
서로 다른 연결에서 처리

연결 처리 흐름



RNetStart() - 초기화 및 워커 생성

전달받은 처리 함수와 워커 개수가 유효한지 확인
각 연결을 보호할 mutex를 최초 한 번 초기화

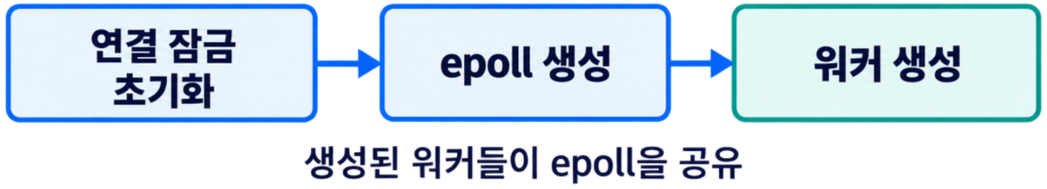
frameHandler에는 이후 완성된 패킷을 처리할 함수를 저장

epoll_create1()로 여러 연결을 감시할 공용 epoll을 생성
pthread_create()로 지정한 수만큼 워커를 생성

모든 워커는 같은 epollFd를 사용

워커 생성에 실패하면 RNetStop()으로 정리

초기화 / 워커 생성




```
int RNetStart(RNetFrameHandler handler, int count)
{
    if(handler == NULL || count < 1 || count > NET_MAX_WORKERS)
    {
        errno = EINVAL;
        return -1;
    }
    if(!netInitialized)
    {
        for(int index = 0; index < NET_MAX_CONNECTIONS; ++index)
        {
            if(pthread_mutex_init(&connections[index].lock, NULL) != 0)
            {
                return -1;
            }
        }
        netInitialized = 1;
    }
    frameHandler = handler;
    netStopping = 0;
    workersStopping = 0;
    epollFd = epoll_create1(EPOLL_CLOEXEC);
    if(epollFd < 0)
    {
        RLOG_ERROR("epoll_create1: %s", strerror(errno));
        return -1;
    }
    for(workerCount = 0; workerCount < count; ++workerCount)
    {
        int createResult = pthread_create(&workers[workerCount], NULL, RunWorker, NULL);

        if(createResult != 0)
        {
            RLOG_ERROR("pthread_create(network worker): %s", strerror(createResult));
            RNetStop();
            errno = createResult;
            return -1;
        }
    }
    return 0;
}
```

RunWorker() — 이벤트 대기 및 처리

워커가 반복 실행하는 함수입니다.

종료 요청을 확인한 뒤 epoll_wait()로 읽기·쓰기 등의 이벤트를 대기

이벤트가 발생하면 HandleConnection()에 연결 정보와 이벤트 종류를 전달

세 번째 인자인 1은 각 호출에서 이벤트를 최대 한 개 핸들링

서버 전체가 연결 하나만 처리한다는 의미는 아님

여러 워커가 서로 다른 연결을 처리하므로, 한 연결의 처리가 오래 걸려도 다른 워커가 다른 연결을 처리

대기는 NET_TICK_MS마다 끝나며, 반복마다 CheckTimeouts()로 시간 초과도 확인

```
static void *RunWorker(void *arg)
{
    (void)arg;
    while(1)
    {
        struct epoll_event event;
        int eventCount;
        int stopping;

        pthread_mutex_lock(&tableMutex);
        stopping = workersStopping;
        pthread_mutex_unlock(&tableMutex);
        if(stopping)
        {
            break;
        }
        // 한 번에 하나만 받음: handler가 오래 걸려도 다른 연결은 다른 워커가 처리
        eventCount = epoll_wait(epollFd, &event, 1, NET_TICK_MS);
        if(eventCount > 0)
        {
            HandleConnection(event.data.u64, event.events);
        }
        else if(eventCount < 0 && errno != EINTR)
        {
            RLOG_ERROR("epoll_wait: %s", strerror(errno));
        }
        CheckTimeouts();
    }
    return NULL;
}
```

연결 상태에 맞춰 이벤트 재등록 (RearmConnection)

연결이 열려 있으면 EPOLLIN으로 수신 가능 상태를 감시

송신 버퍼에 데이터가 남아 있으면 EPOLLOUT도 추가,
소켓이 쓰기 가능한 상태가 되었을 때 남은 데이터를 보냄

EPOLLONESHOT은 이벤트를 한 번 전달하면 해당 연결의 이벤트 감시를 비활성화

따라서 처리가 끝난 뒤 EPOLL_CTL_MOD로 다음 이벤트를 받을 수 있도록 다시 활성화

같은 연결의 중복 처리는 연결 소유 상태와 mutex도 함께 사용해 관리

```
static void RearmConnection(RNetConnection *connection)
{
    struct epoll_event event = {.data.u64 = MakeEventData(connection)};
    uint32_t interest = 0;

    if(connection->state == NET_STATE_OPEN)
    {
        interest = EPOLLIN;
    }
    if(connection->txLength > 0)
    {
        interest |= EPOLLOUT;
    }
    event.events = interest | EPOLLONESHOT;
    epoll_ctl(epollFd, EPOLL_CTL_MOD, connection->fd, &event);
}
```

이벤트 재등록



동적 파라미터 기반 쿼리 실행

SQL 형식은 QUERY_*로 정의
실행 시 입력값을 ?에 동적으로 바인딩

DATABASE_TEXT .• 문자열 값
DATABASE NUMBER : 정수 값

공통 실행 함수
ExecuteDatabaseQuery : 실행 및 행별 결과 전달
QueryDatabaseValue : 첫 행•첫 열 값 조회

```
// ? = id
#define QUERY_SELECT_MEMBER_PASSWORD "SELECT pw_hash FROM member WHERE id = ? LIMIT 1"
// ? = id
#define QUERY_SELECT_MEMBER "SELECT id, type FROM member WHERE id = ? LIMIT 1"
// ? = id
#define QUERY_SELECT_MEMBER_TYPE "SELECT type FROM member WHERE id = ? LIMIT 1"
#define QUERY_SELECT_MEMBER_ALL "SELECT id, type FROM member ORDER BY id"
// ? = id, pw_hash, type
#define QUERY_INSERT_MEMBER "INSERT INTO member(id, pw_hash, type) VALUES(?, ?, ?)"
// ? = pw_hash, id
#define QUERY_UPDATE_MEMBER_PASSWORD "UPDATE member SET pw_hash = ? WHERE id = ?"
// ? = id (bluetooth, dht 행은 FK로 함께 삭제)
#define QUERY_DELETE_MEMBER "DELETE FROM member WHERE id = ?"

typedef struct _DatabaseValue
{
    const char *text;
    long long number;
} DatabaseValue;


speedParam = DATABASE_NUMBER(data.fanSpeed);
if(ExecuteDatabaseQuery(QUERY_UPDATE_FAN, &speedParam, 1, NULL, NULL, NULL) != 0)
{
    RLOG_WARN("[%s] FAN UPDATE failed: database UPDATE, speed=%u", session->label,
        (unsigned int)data.fanSpeed);
    return RPacketSendAck(session, REQ_FAN_UPDATE, 0);
}
queryResult = QueryDatabaseValue(QUERY_SELECT_FAN, NULL, 0, speedText, sizeof(speedText));
if(queryResult != 1)
{
    RLOG_WARN("[%s] FAN UPDATE verification failed: singleton_id=1, lookup=%d", session->label,
        queryResult);
    return RPacketSendAck(session, REQ_FAN_UPDATE, 0);
}
```

트러블 슈팅

펌웨어 업데이트 - 부트로더 적용 후 부팅 실패

증상

부트로더 적용 후 정상 부팅 실패 (벽돌 현상)

원인

앱 주소 및 섹터 위치 불일치로 인한 잘못된 접근

해결

앱 주소 / VTOR 정렬

안전한 MSP 전환

```
MEMORY
{
RAM (xrw)      : ORIGIN = 0x20000000, LENGTH = 128K
/* 섹터 0 = 부트로더(Bootloader/), 섹터 1~4 (0x08004000, 112K) = 앱.
   섹터 5 = 펌웨어 업데이트 대기 영역, 섹터 7 = SData. 배치는 common/IoTFirmware.h 참고 */
FLASH (rx)      : ORIGIN = 0x8004000, LENGTH = 112K
}

# SystemInit(system_stm32f4xx.c)은 STM32_Drivers 라이브러리에서 따로 컴파일되므로
여기에도 넣어야 VTOR이 옮겨짐
# (없으면 디버거로 앱을 바로 실행할 때 SysTick이 0x08000000 벡터로 가서 멈춤,
cmake/stm32cubemx는 CubeMX가 덮어씀)
target_compile_definitions(STM32_Drivers PRIVATE
    USER_VECT_TAB_ADDRESS
    VECT_TAB_OFFSET=0x00004000U


SCB->VTOR = FIRMWARE_APP_ADDRESS;
__DSB();
__ISB();
__asm volatile("msr msp, %0\n\tbx %1" : :
    "r"(stackPointer), "r"(resetHandler) : "memory");
__builtin_unreachable();
```

하드 코딩으로 인한 비인가 유저 접속 문제

증상
비인가 사용자가 고정 ID / PW 로 로그인

원인
펌웨어에 ID / PW를 직접 기입으로 인한 외부 유출

해결
플래시 메모리에 CLI을 활용한 ID / PW 업로드

1 CLI로 장치별 계정 설정
member set <id> <PW>
member show
member clear

코드 리빌드 없이 계정 설정 변경

2 장치의 비휘발성 메모리에 저장
// STM32: Flash

SDataSave(SDATA ADDR_MEMBER,
(const uint8_t *)&member, sizeof(member));

// Arduino: EEPROM

SaveData(DATA_ADDR MEMBER,
reinterpret_cast<const uint8 t *>(&member),
sizeof(member));

역할 및 마무리



변한빛

- Raspberry 서버 아키텍쳐 구현
- Raspberry,STM32,Arduino CLI 구현
- 통신 프로토콜 설계
- STM32 모듈 별 드라이버 개발
- Arduino 모듈 별 드라이버 개발
- 초기 환경 설정



나재우

- Raspberry 서버 프로토타입 구현
- Raspberry CLI 프로토타입 구현
- 통신 프로토콜 프로토 타입 구현
- PC 소프트웨어 GUI
- PC 소프트웨어 기능 구현
- ADht11 드라이버 구현
- PPT 프로토타입 구현

Q&A



질의응답 시간입니다.